

Software Security Analysis

Write C programs and an assembly Program

Introduction

This lab is designed to work with both C program and assembly language to understand the fundamental concepts of systems programming. C programming allows developers to write readable and structured code and assembly language provides direct control over hardware execution. The flowchart is used to visually represent the logical flow of a pass/fail grading program. A flowchart is a pictorial representation of an algorithm in which the steps are drawn in the form of different shapes of boxes and the logical flow is indicated by interconnecting arrows (Domas, S., & Domas, C., 2024).

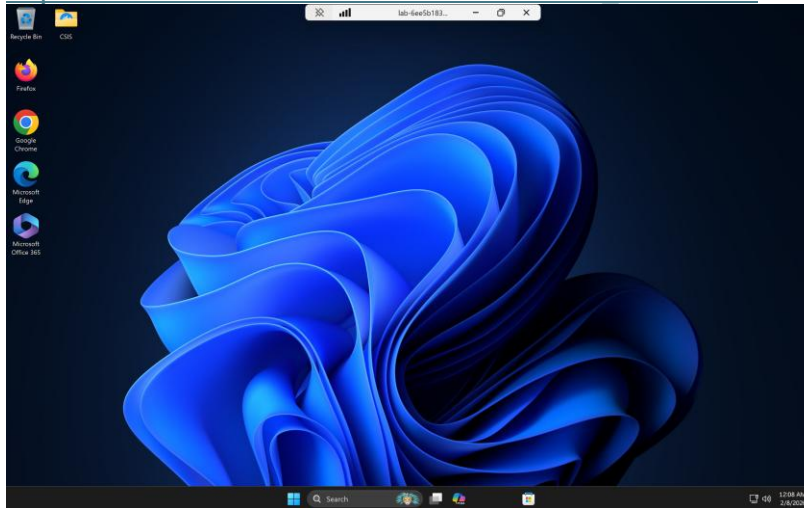
Then implemented the pass/fail grading program using both an integrated development environment and Ubuntu running under Windows Subsystem for Linux (WSL). The program includes functions, conditional statements, and input validation. Then the same program was implemented in assembly language. It gives the idea of how compilers translate high-level instructions into machine-level operations.

As a final step, a simple Hello World program is disassembled using *objdump* tool. It shows how stack frames, function calls, and program termination are handled at the machine level.

Flowchart and write a C program to assist in awarding grades in a Pass/Fail class

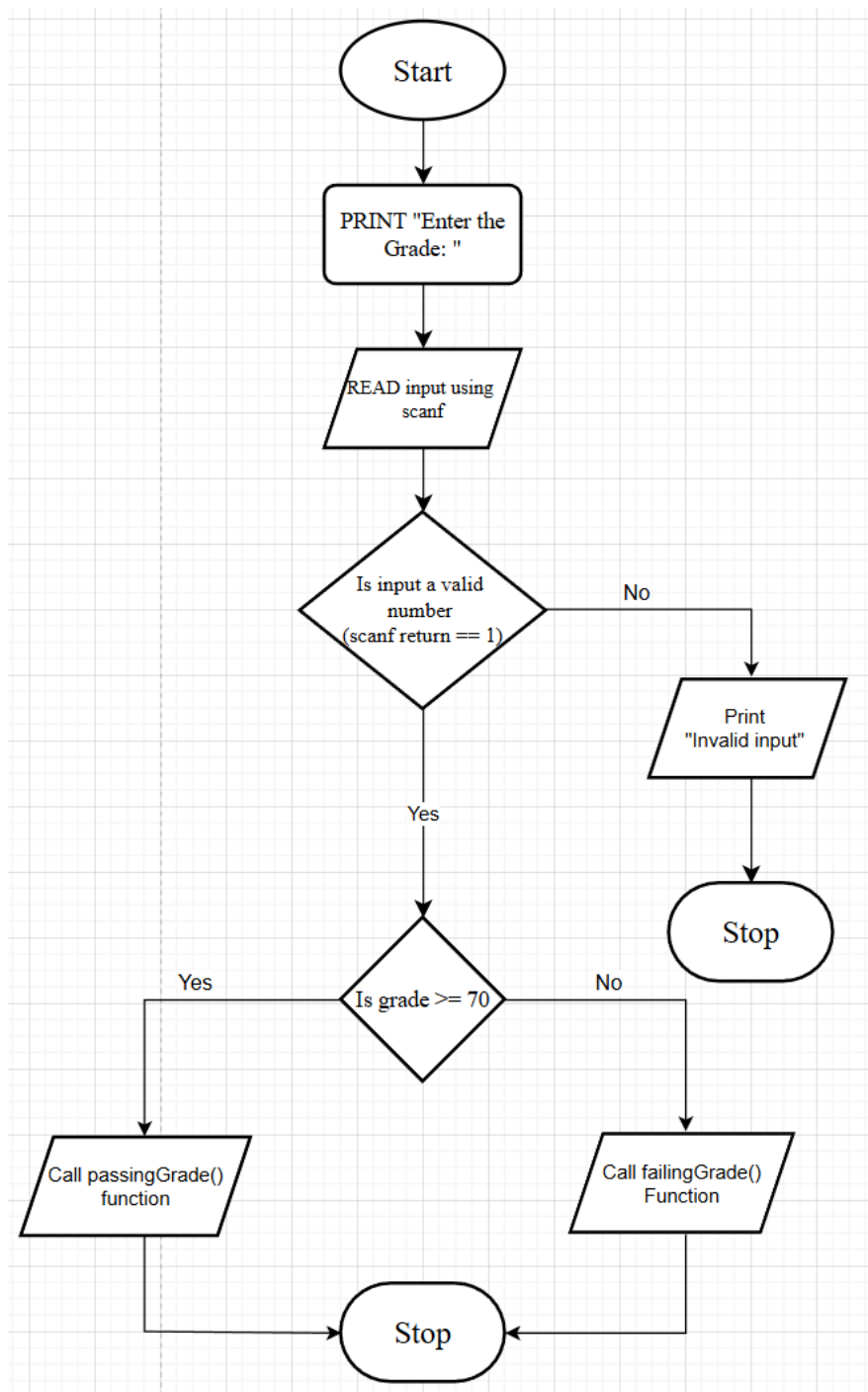
1. Logged to Azure Lab environment using following web link and logged into the Windows VM using Oracle VirtualBox.

https://labs.azure.com/virtualmachines?feature_vnext=true



2. Created a flowchart using <https://www.drawio.com/> website.
 - Used an *Ellipse* to *Start* the flowchart.
 - Used a *Rectangle* to display the prompt “*Enter the Grade:*” to the user.
 - Used a *Parallelogram* to read the grade input using *scanf*.
 - Used a *Diamond* to check whether the input is a valid number (*scanf* return value = 1).

- Used a *Parallelogram* to print "Invalid input" when the input is not a valid number.
- Used a *Terminator* to stop the flowchart after an invalid input.
- Used a *Diamond* to evaluate whether the grade is greater than or equal to 70.
- Used a *Parallelogram* to call the *passingGrade()* function when the grade is greater than or equal to 70.
- Used a *Parallelogram* to call the *failingGrade()* function when grade is less than 70.
- Used a *Terminator* to represent the Stop of the flowchart.



3. Wrote a C program using Code::Blocks to display to assist in awarding grades in a Pass/Fail class.

```
Start here X HashanK.c X MyFirstCProgram.c X SSADiscussion.c X
1 #include <stdio.h>
2
3 //create function to print passing grade
4 void passingGrade () {
5     printf("This is a passing grade.\n");
6 }
7
8 //create function to print failing grade
9 void failingGrade(){
10     printf("This is a failing grade.\n");
11 }
12
13 int main() {
14     int grade; // declare integer variable to store the grade
15
16     printf("Enter the Grade: "); // Prompt to enter a grade to the user
17
18     if (scanf("%d", &grade) !=1) { // Read input and Check if input is a valid number
19         printf("Invalid input,Please enter a valid number. \n"); // If its not a number displays an error message
20         return 0; //exit program
21     }
22
23     if (grade >= 70) {
24         passingGrade(); // call the function passingGrade
25     }
26     else {
27         failingGrade(); // call the function failingGrade
28     }
29     return 0; //exit program
30 }
```

4. Program was compiled and executed using Code::Blocks and displayed output for grade is 70.

```
"E:\2. Moorhead University\C X + v
Enter the Grade: 70
This is a passing grade.

Process returned 0 (0x0)   execution time : 5.693 s
Press any key to continue.
```

5. Program was compiled and executed using Code::Blocks and displayed output for grade is below 70.

```
"E:\2. Moorhead University\C X + v
Enter the Grade: 58
This is a failing grade.

Process returned 0 (0x0)   execution time : 2.685 s
Press any key to continue.
```

6. Program was compiled and executed using Code::Blocks and displayed output for grade as a letter.

```
"E:\2. Moorhead University\C X + v
Enter the Grade: k
Invalid input,Please enter a valid number.

Process returned 0 (0x0)   execution time : 5.629 s
Press any key to continue.
```

Part a – write a C program for awarding grades in a Pass/Fail class using Ubuntu

1. Open PowerShell and tried to install WSL, but it is already installed.

```
hashan@lab8K4EB1: ~  
Windows PowerShell  
Copyright (C) Microsoft Corporation. All rights reserved.  
  
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows  
  
PS C:\Users\CSIS-Admin> wsl --install  
Ubuntu is already installed.  
Launching Ubuntu...  
To run a command as administrator (user "root"), use "sudo <command>".  
See "man sudo_root" for details.  
  
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 5.15.167.4-microsoft-standard-WSL2 x86_64)  
  
* Documentation: https://help.ubuntu.com  
* Management: https://landscape.canonical.com  
* Support: https://ubuntu.com/pro  
  
System information as of Sat Feb 7 20:39:45 CST 2026  
  
System load: 0.38 Processes: 55  
Usage of /: 0.1% of 1006.85GB Users logged in: 0  
Memory usage: 5% IPv4 address for eth0: 172.21.138.223  
Swap usage: 0%  
  
This message is shown once a day. To disable it please create the  
/home/hashan/.hushlogin file.  
hashan@lab8K4EB1:~$
```

2. Updated Ubuntu and created a C source file HashanK.c

```
hashan@lab8K4EB1:~$ sudo apt update  
[sudo] password for hashan:  
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease  
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]  
hashan@lab8K4EB1:~$ sudo nano HashanK.c
```

3. Wrote a C program to assist in awarding grades in a Pass/Fail class.

```
GNU nano 7.2 HashanK.c *  
#include <stdio.h>  
  
//create function to print passing grade  
void passingGrade () {  
    printf("This is a passing grade.\n");  
}  
  
//create function to print failing grade  
void failingGrade(){  
    printf("This is a failing grade.\n");  
}  
  
int main() {  
    int grade; // declare integer variable to store the grade  
  
    printf("Enter the Grade: "); // Prompt to enter a grade to the user  
  
    if (scanf("%d", &grade) !=1) { // Read input and Check if input is a valid number  
        printf("Invalid input,Please enter a valid number. \n"); // If its not a number displays an error message  
        return 0; //exit program  
    }  
  
    if (grade >= 70) {  
        passingGrade(); // call the function passingGrade  
    }  
    else {  
        failingGrade(); // call the function failingGrade  
    }  
    return 0; //exit program  
}  
  
File Name to Write: HashanK.ces  
^G Help M-D DOS Format M-A Append M-B Backup File  
^C Cancel M-M Mac Format M-P Prepend ^T Browse
```

4. Compiled the program, but GNU C Compiler is not installed.

```
hashan@lab8K4EB1:~$ gcc HashanK.c -o HashanK.out
Command 'gcc' not found, but can be installed with:
sudo apt install gcc
hashan@lab8K4EB1:~$ sudo apt install gcc
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
```

5. Installed the GNU C Compiler and compile the program. Then verify the executable file.

```
hashan@lab8K4EB1:~$ gcc HashanK.c -o HashanK.out
hashan@lab8K4EB1:~$ ls -l
total 20
-rw-r--r-- 1 root root 816 Feb 7 20:48 HashanK.c
-rwxr-xr-x 1 hashan hashan 16176 Feb 7 20:51 HashanK.out
hashan@lab8K4EB1:~$
```

6. The program was tested using a grade value below 70, a grade value 70, and a non-numeric (letter) input to verify correct behavior.

```
hashan@lab8K4EB1:~$ ./HashanK.out
Enter the Grade: 65
This is a failing grade.
hashan@lab8K4EB1:~$ ./HashanK.out
Enter the Grade: 70
This is a passing grade.
hashan@lab8K4EB1:~$ ./HashanK.out
Enter the Grade: a
Invalid input, Please enter a valid number.
hashan@lab8K4EB1:~$
```

Step b - Write an *assembly* program for Part 1a

1. Created the assembly source file HashanK.asm

```
hashan@lab8K4EB1:~$ sudo nano HashanK.asm
```

2. Wrote an assembly program for Part 1.
 - Displays a prompt
 - Reads user input
 - Converts ASCII digits to an integer
 - Validates numeric input
 - Compares the grade against 70
 - Prints the appropriate message

```
GNU nano 7.2 HashanK.asm
section .data
    prompt    db "Enter the Grade: "      ; Prompt shown to the user
    promptLen equ $ - prompt              ; Length of the prompt string

    passMsg   db "This is a passing grade.", 10 ; Message for passing grade
    passLen   equ $ - passMsg              ; Length of passing message

    failMsg   db "This is a failing grade.", 10 ; Message for failing grade
    failLen   equ $ - failMsg              ; Length of failing message

    invalidMsg db "Invalid input. Please enter a valid number.", 10
    invalidLen equ $ - invalidMsg          ; Length of invalid input message

section .bss
    buffer    resb 8      ; Buffer to store user input (ASCII characters)
    grade     resd 1      ; Variable to store converted integer grade

section .text
    global _start        ; Entry point for the Linux program

_start:
    ; Print the prompt "Enter the Grade: "
    ; -----
    mov rax, 1           ; System call number 1 = sys_write
    mov rdi, 1           ; File descriptor 1 = stdout
    mov rsi, prompt      ; Address of the prompt string
    mov rdx, promptLen   ; Number of bytes to write
    syscall              ; Invoke kernel to print prompt

    ; Read user input from keyboard
    ; -----
    mov rax, 0           ; System call number 0 = sys_read
    mov rsi, buffer       ; Address where input will be stored
    mov rdx, 8           ; Maximum number of bytes to read
    syscall              ; Invoke kernel to read input

    ; Convert ASCII input into an integer
    ; -----
    xor eax, eax         ; Clear EAX register (grade = 0)
    xor ecx, ecx         ; Clear ECX register (index = 0)

convert_loop:
```

```

convert_loop:
    mov bl, [buffer + ecx]    ; Load one character from input buffer
    cmp bl, 10                ; Compare character with newline ('\n')
    je conversion_done        ; If newline, stop conversion

    cmp bl, '0'               ; Check if character is less than '0'
    jl invalid_input          ; If yes, input is invalid
    cmp bl, '9'               ; Check if character is greater than '9'
    jg invalid_input          ; If yes, input is invalid

    sub bl, '0'               ; Convert ASCII digit to numeric value
    imul eax, eax, 10          ; Multiply current grade by 10
    add eax, ebx               ; Add new digit to grade

    inc ecx                   ; Move to next character
    jmp convert_loop          ; Repeat conversion loop

conversion_done:
    mov [grade], eax          ; Store final integer grade value

    ; Check if grade is greater than or equal to 70
    ; -----
    cmp eax, 70               ; Compare grade with passing threshold
    jl failing_grade          ; If grade < 70, jump to failing grade

passing_grade:
    mov rax, 1                ; sys_write
    mov rdi, 1                ; stdout
    mov rsi, passMsg          ; Address of passing message
    mov rdx, passLen          ; Length of passing message
    syscall                   ; Print passing message
    jmp exit_program          ; Exit program

failing_grade:
    mov rax, 1                ; sys_write
    mov rdi, 1                ; stdout
    mov rsi, failMsg          ; Address of failing message
    mov rdx, failLen          ; Length of failing message
    syscall                   ; Print failing message
    jmp exit_program          ; Exit program

invalid_input:
    mov rax, 1                ; sys_write
    mov rdi, 1                ; stdout
    mov rsi, invalidMsg        ; Address of invalid input message
    mov rdx, invalidLen        ; Length of invalid input message
    syscall                   ; Print error message

```

File Name to Write: HashanK.asm

^G Help

^C Cancel

M-D

DOS Format

M-M

Mac Format

M-A

Append

M-P

Prepend

3. Tried to assemble the program, but *nasm* is not installed. Then installed the *nasm*.

```
hashan@lab8K4EB1:~$ nasm -f elf64 -o HashanK.o HashanK.asm
```

Command 'nasm' not found, but can be installed with:

```
sudo apt install nasm
```

```
hashan@lab8K4EB1:~$ sudo apt install nasm
```

```
Reading package lists... Done
```

```
Building dependency tree... Done
```

```
Reading state information... Done
```

```
The following NEW packages will be installed:
```


4. Assembled the program and ran *ls* to verify creation of the object file.

```
hashan@lab8K4EB1:~$ nasm -f elf64 -o HashanK.o HashanK.asm
hashan@lab8K4EB1:~$ ls -l
total 28
-rw-r--r-- 1 root root 2297 Feb 7 21:05 HashanK.asm
-rw-r--r-- 1 root root 816 Feb 7 20:48 HashanK.c
-rw-r--r-- 1 hashan hashan 1776 Feb 7 21:08 HashanK.o
-rwxr-xr-x 1 hashan hashan 16176 Feb 7 20:51 HashanK.out
hashan@lab8K4EB1:~$
```

5. Linked the object file to the executable by running *ld -o HashanK HashanK.o* command. Then ran the program using *./HashanK* command and capture the results.

```
hashan@lab8K4EB1:~$ ld -o HashanK HashanK.o
hashan@lab8K4EB1:~$ ./HashanK
Enter the Grade: 65
This is a failing grade.
hashan@lab8K4EB1:~$ ./HashanK
Enter the Grade: 75
This is a passing grade.
hashan@lab8K4EB1:~$ ./HashanK
Enter the Grade: w
Invalid input. Please enter a valid number.
hashan@lab8K4EB1:~$
```

The program was tested using a grade value below 70, a grade value above 70, and a non-numeric (letter) input to verify correct pass, fail, and invalid input handling.

Step c - Write a hello world program in C and disassemble to see the machine code version

1. Created the Hello World source file *hello.c*

```
hashan@lab8K4EB1:~$ sudo nano hello.c
```

2. Wrote a Hello World C program.

```
hashan@lab8K4EB1: ~
GNU nano 7.2 hello.c *
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

3. Then compiled *hello.c* and produces the executable file *hello.out*.

```
hashan@lab8K4EB1:~$ gcc hello.c -o hello.out
hashan@lab8K4EB1:~$
```

4. Disassembled the compiled binary by running `objdump -M intel -D hello.out | grep -A20 main.:` command. The command displayed 20 lines after the regular expression, `main.`

```
hashan@lab8k4EB1:~$ objdump -M intel -D hello.out | grep -A20 main.:
0000000000001149 <main>:
1149:    f3 0f 1e fa    endbr64
114d:    55            push    rbp
114e:    48 89 e5      mov     rbp, rsp
1151:    48 8d 05 ac 0e 00 00    lea     rax, [rip+0xeac]    # 2004 <_IO_stdin_used+0x4>
1158:    48 89 c7      mov     rdi, rax
115b:    e8 f0 fe ff ff    call   1050 <puts@plt>
1160:    b8 00 00 00 00    mov     eax, 0x0
1165:    5d            pop     rbp
1166:    c3            ret

Disassembly of section .fini:

0000000000001168 <_fini>:
1168:    f3 0f 1e fa    endbr64
116c:    48 83 ec 08    sub     rsp, 0x8
1170:    48 83 c4 08    add     rsp, 0x8
1174:    c3            ret

Disassembly of section .rodata:

hashan@lab8k4EB1:~$
```

`objdump -D` disassembles the binary into assembly instructions. `-M intel` displays instructions in Intel syntax and `grep -A20 main.:` shows the main function and 20 lines after it.

5. I selected following lines from the disassembled code. The disassembled output shows how the C Hello World program is translated into Intel x86-64 instructions, including stack frame setup, a function call to print output, and a return to the operating system.

```
114d:    55            push    rbp
114e:    48 89 e5      mov     rbp, rsp
1151:    48 8d 05 ac 0e 00 00    lea     rax, [rip+0xeac]    # 2004 <_IO_stdin_used+0x4>
```

55 *push rbp*

The instruction `push rbp` (machine code 0x55) is a fundamental part of the function opening in x86-64 assembly. It is used to save base pointer of the caller function onto the stack, allowing the current function to use `rbp` to manage its own stack frame.

48 89 e5 *mov rbp, rsp*

This instruction sets up the stack frame for `main` by copying the current stack pointer, `rsp` into the base pointer, `rbp`. This allows the function to reference local variables and manage function calls reliably.

48 8d 05 ac 0e 00 00 *lea rax, [rip+0xeac]*

This instruction loads an effective memory address into the `rax` register. The address is calculated relative to the instruction pointer, `rip` and points to the location of the string "Hello, World!" in memory. This address is later passed as an argument to the `puts` function for printing.

Conclusion

This lab helps to cover programming concepts starting from flowchart design to C programming and low-level assembly language implementation. A clearly structured flowchart provides the proper handling of program logic and contributes to accurate output generation. Implementing the pass/fail grading logic in C highlighted fundamental programming concepts such as conditional statements, function calls, and user input validation. Compiling and executing our program using both Code::Blocks and Ubuntu using WSL demonstrated practical and real-world development workflows.

The assembly language implementation showed the complexity of even a small program, as multiple instructions are required to perform simple operations. Further, the disassembly of the Hello World program showed the way of actual machine instructions executed by the CPU. Combination of these steps show how the program execution, compiler translation, and the layered nature of computer systems.

References

1. Domas, S., & Domas, C. (2024). *x86 software reverse-engineering, cracking, and counter-measures*. Wiley.
2. W3Schools. (n.d.). *C Tutorial*. <https://www.w3schools.com/c/index.php>
3. Code::Blocks. (n.d.). *Downloads — Code::Blocks*.
<https://www.codeblocks.org/downloads/binaries/>
4. diagrams.net. (n.d.). *draw.io – Free online diagram software*. <https://www.drawio.com/>
5. Lucid Software Inc. (n.d.). *What is a flowchart? Flowchart tutorial*.
<https://www.lucidchart.com/pages/what-is-a-flowchart-tutorial>
6. Microsoft. (n.d.). *How to use the Disassembly window* | Microsoft Learn. Microsoft. Retrieved February 7, 2026, from <https://learn.microsoft.com/en-us/visualstudio/debugger/how-to-use-the-disassembly-window?view=visualstudio>
7. Reese, J. (n.d.). *Understanding C by learning assembly*. Recurse Center. Retrieved February 7, 2026, from <https://www.recurse.com/blog/7-understanding-c-by-learning-assembly>